LLM Interview Questions & Answers

Kalyan KS

100+ questions with detailed explanations

LLM Interview Questions and Answers

Author: Kalyan KS

Contents

LLM Interview Questions and Answers

First Edition

Published: January 2026

Author: Kalyan KS, NLP Researcher and Consultant. **(LinkedIn** and **Twitter)**

Disclaimer

The information in this book is distributed on an "As Is" basis, without warranty. While every precaution has been taken in the preparation of this work, the author shall not have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in this book.

Question 1: CNNs and RNNs don't use positional embeddings. Why do transformers use positional embeddings?

Answer

CNNs and RNNs don't use positional embeddings because these models inherently capture positional information through convolutional filters or recurrence over time steps. However, the Transformer model uses the self-attention mechanism, which processes all tokens in parallel without any notion of sequence or order.

Positional embeddings inject explicit information about each token's position in the sequence, allowing transformers to understand word order and relative distances between tokens. This helps them model sequential dependencies and sentence structure effectively, compensating for the parallel and order-agnostic nature of self-attention mechanisms.

Detailed Explanation

Take the sentence:

> "Cats chase mice."

If we shuffle the words:

> "Mice chase cats."

The meaning completely changes. This simple example shows why any language model must understand **word order**, not just the words themselves.

Why RNNs Don't Need Positional Embeddings

An RNN (Recurrent Neural Network) processes a sentence sequentially, reading one word at a time from left to right. For example, given the sentence "I love apples," the model first reads "I," then "love," and finally "apples."

- Step 1 → "I"
- Step 2 → "love"
- Step 3 → "apples"

Because each word is processed after the previous one, the model inherently knows that "I" came before "love," and "love" came before "apples." This sequential processing gives RNNs a natural understanding of order, which is why they do not require positional embeddings.

Why CNNs Don't Need Positional Embeddings

A CNN (Convolutional Neural Network) processes text using sliding windows that move across the sentence. Consider the sentence "The dog barked loudly." A convolutional window might first capture "The dog barked" and then slide to capture "dog barked loudly."

- Window 1 → "The dog barked"
- Window 2 → "dog barked loudly"

These windows preserve local word order—"dog" appears before "barked," and "barked" appears before "loudly." By learning patterns within these overlapping windows, CNNs implicitly learn positional relationships, making explicit positional embeddings unnecessary.

Why Transformers Are Different

Transformers operate very differently. They process all words in a sentence at the same time, rather than sequentially or through sliding windows. For the sentence "She ate cake," a Transformer sees the tokens "She," "ate," and "cake" in parallel, without any inherent order. To the self-attention mechanism, this can look like an unordered set of words. If the words are shuffled, the model would not automatically know that the meaning has changed.

Why Transformers Need Positional Embeddings

Because self-attention is order-agnostic, a Transformer has no built-in sense of which word came first, which came next, or how far apart words are. Positional embeddings solve this problem by adding position information to each word's representation.

Word	Position	Positional embedding meaning
She	1	"I'm the first word."
ate	2	"I'm after She."
cake	3	"I'm the third word."

These positional embeddings are added to the token embeddings:

Token Embedding (Meaning) + Positional Embedding (Position) = Final embedding

With this additional information, the Transformer can correctly interpret that "She" comes before "ate," and "ate" comes before "cake." As a result, it can distinguish between "She ate cake" and "Cake ate she." .

Question 2: Tell me the basic steps involved in running an inference query on an LLM.

Answer

Running an inference query on an LLM involves the following steps:

- Tokenization: The input text is first broken down into tokens, and then the tokens are mapped to their IDs in the model's vocabulary.
- Prefill Phase: The embedding layer converts these token IDs into embeddings, and then the decoder layers process these embeddings simultaneously to compute intermediate states (keys and values). This parallel processing establishes the context for generating new tokens.
- Decoding Phase: The model generates output tokens one at a time autoregressively, each based on previously generated tokens and cached states, continuing until a stopping condition is met.
- Detokenization: Finally, the generated tokens are converted back into human-readable text.

Detailed Explanation

Tokenization — Breaking Text into Small Pieces

Before a large language model (LLM) can understand any text, the text must first be broken down into smaller units called *tokens*. A token can be a full word, part of a word, or even a punctuation mark. For example, consider the input text:

> **"Hello world!"**

A tokenizer might split this text into the following tokens:

- "Hello"
- " world"
- "!"

Each of these tokens is then mapped to a numerical ID, since all internal processing is numerical.

Example token IDs:

- "Hello" → 1564

- " world" → 982
- "!" → 299

After tokenization, the original text is represented numerically as:

[1564, 982, 299]

This numerical sequence is what is actually fed into the model.

Prefill Phase — The Model Reads the Entire Input at Once

Once tokenization is complete, the model enters what is known as the *prefill phase*. In this phase, the LLM processes the entire input sequence at the same time, effectively "reading" the full sentence in one pass. This phase involves the following steps:

- Each token ID is converted into an embedding, which is a vector of numbers representing semantic meaning.
- All input tokens are processed in parallel, rather than sequentially.
- The model constructs a memory-like structure using keys and values, commonly referred to as the **KV cache**.

These keys and values represent what the model has understood from the input so far and will be reused during generation to avoid recomputing past information.

Decoding Phase — The Model Generates One Token at a Time

After the input has been fully understood, the model moves to the *decoding phase*, which is where the actual response is generated. Unlike the prefill phase, decoding happens sequentially.

Consider the question:

"What is 2 + 2?"

The model might generate the following tokens, one after another:

- "The"
- " answer"
- " is"
- " 4"

Detokenization — Converting Tokens Back Into Text

Once the model has finished generating tokens, the final step is *detokenization*. This step converts numerical token IDs back into human-readable text.

For example:

> Generated token IDs: [**1011, 938, 117, 21**]
>
> Detokenized output: **"The answer is 4"**

This detokenized text is what the user ultimately sees as the model's response.